

Text Data in Business and Economics
University of Basel, 21–25 September 2026
Benjamin W. Arolt, University of Cambridge

4. Document Distance

Outline

Document Distance

Preview: Retrieval-Augmented Generation

Clustering

Document Distance/Proximity

- ▶ In economics, we often want to compare documents to one another
- ▶ Example, how close is a political speech to the party leader?

Document Distance/Proximity

- ▶ In economics, we often want to compare documents to one another
- ▶ Example, how close is a political speech to the party leader?
- ▶ Today, we will focus on methods designed to measure document distance/proximity
- ▶ But almost everything we do in this class can be framed as measuring document distance in some way

Text Re-Use

- ▶ Text Re-Use algorithms (like “Smith-Waterman” or BLAST) measure similarity by finding and counting shared sequences in two texts above some minimum length, e.g. 5 words
 - ▶ useful for plagiarism detection, for example
- ▶ precise but slow
 - ▶ shortcut: look at proportion of shared (hashed) 5-grams across texts

Document-Term Matrix

The **document-term matrix $\mathbf{X}$** :

- ▶ each row d represents a **document**, while each column w represents a word (or term more generally, e.g. n-grams)
 - ▶ A matrix entry $\mathbf{X}_{[d,w]}$ quantifies the strength of association between a document and a word, generally its count or frequency

Document-Term Matrix

The **document-term matrix** $\mathbf{X}$:

- ▶ each row d represents a **document**, while each column w represents a word (or term more generally, e.g. n-grams)
 - ▶ A matrix entry $\mathbf{X}_{[d,w]}$ quantifies the strength of association between a document and a word, generally its count or frequency
- ▶ each document/row $\mathbf{X}_{[d,:]}$ is a distribution over terms
 - ▶ these vectors have a **spatial interpretation** → geometric distances between document vectors reflect semantic distances between documents in terms of shared terms

Document-Term Matrix

The **document-term matrix $\mathbf{X}$** :

- ▶ each row d represents a **document**, while each column w represents a word (or term more generally, e.g. n-grams)
 - ▶ A matrix entry $\mathbf{X}_{[d,w]}$ quantifies the strength of association between a document and a word, generally its count or frequency
- ▶ each document/row $\mathbf{X}_{[d,:]}$ is a distribution over terms
 - ▶ these vectors have a **spatial interpretation** → geometric distances between document vectors reflect semantic distances between documents in terms of shared terms
- ▶ each word/column $\mathbf{X}_{[:,w]}$ is a distribution over documents
 - ▶ these vectors also have a spatial interpretation! geometric distances between word vectors reflect semantic distances between words in terms of showing up in the same documents

Cosine Similarity

- ▶ Each document is a vector x_d , e.g. term counts or TF-IDF frequencies
- ▶ → Each document is a non-negative vector in an n_x -space, where $n_x =$ vocabulary size
 - ▶ that is, documents are rays, and similar documents have similar vectors

Cosine Similarity

- ▶ Each document is a vector x_d , e.g. term counts or TF-IDF frequencies
- ▶ → Each document is a non-negative vector in an n_x -space, where n_x = vocabulary size
 - ▶ that is, documents are rays, and similar documents have similar vectors
- ▶ Can measure similarity between documents i and j by the cosine of the angle between x_i and x_j :
 - ▶ With perfectly collinear documents (that is, $x_i = \alpha x_j$, $\alpha > 0$), $\cos(0) = 1$
 - ▶ For orthogonal documents (no words in common), $\cos(\pi/2)=0$

Cosine Similarity

- ▶ Each document is a vector x_d , e.g. term counts or TF-IDF frequencies
- ▶ → Each document is a non-negative vector in an n_x -space, where n_x = vocabulary size
 - ▶ that is, documents are rays, and similar documents have similar vectors
- ▶ Can measure similarity between documents i and j by the cosine of the angle between x_i and x_j :
 - ▶ With perfectly collinear documents (that is, $x_i = \alpha x_j$, $\alpha > 0$), $\cos(0) = 1$
 - ▶ For orthogonal documents (no words in common), $\cos(\pi/2)=0$

Cosine similarity is computable as the normalized dot product between the vectors:

$$\cos_sim(x_1, x_2) = \frac{x_1 \cdot x_2}{||x_1|| ||x_2||}$$

```
from sklearn.metrics.pairwise import
cosine_similarity
# between two vectors:
sim = cosine_similarity(x, y)[0,0]
# between all rows of a matrix:
sims = cosine_similarity(X)
```

Notes on Cosine Similarity

- ▶ For a corpus with n rows, the pairwise similarities give $n \times (n - 1)$ similarity scores

Notes on Cosine Similarity

- ▶ For a corpus with n rows, the pairwise similarities give $n \times (n - 1)$ similarity scores
- ▶ tf-idf down-weights terms that appear in many documents, usually gives better results
 - ▶ tf-idf similarity is the workhorse, used for example in bm25 and elasticsearch

Notes on Cosine Similarity

- ▶ For a corpus with n rows, the pairwise similarities give $n \times (n - 1)$ similarity scores
- ▶ tf-idf down-weights terms that appear in many documents, usually gives better results
 - ▶ tf-idf similarity is the workhorse, used for example in bm25 and elasticsearch

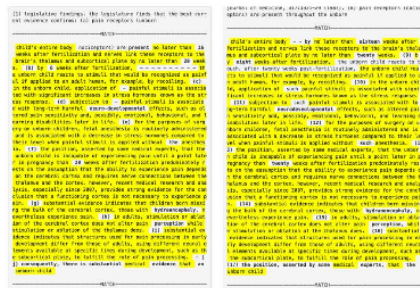
Alternative distance metrics:

- ▶ dot product and Euclidean distance (too sensitive to document length)
- ▶ Jensen-Shannon Divergence
- ▶ Jaccard distance
- ▶ etc.

Burgess et al, “Legislative Influence Detectors”

- ▶ Compare bill texts across states in two-step process:
 - (1) find candidates using elasticsearch (tf-idf similarity);
 - (2) compare candidates using text reuse score.

Burgess et al, “Legislative Influence Detectors”



ABSTRACT

State legislatures introduce at least 45,000 bills each year. However, we lack a clear understanding of who is actually writing those bills. As legislators often lack the time and staff to draft each bill, they frequently copy text written by other states or interest groups.

However, existing approaches to detect text reuse are slow, biased, and incomplete. Journalists or researchers who want to know where a particular bill originated must perform a largely manual search. Watchdog organizations even hire armies of volunteers to monitor legislation for matches. Given the time-consuming nature of the analysis, journalists and researchers tend to limit their analysis to a subset of topics (e.g. abortion or gun control) or a few interest groups.

This paper presents the Legislative Influence Detector (LID). LID uses the Smith-Waterman local alignment algorithm to detect sequences of text that occur in model legislation and state bills. As it is computationally too expensive to run this algorithm on a large corpus of data, we use a search engine built using Elasticsearch to limit the number of comparisons. We show how LID has found 45,405 instances of bill-to-bill text reuse and 14,137 instances of model-legislation-to-bill text reuse. LID reduces the time it takes to manually find text reuse from days to seconds.

1. What is the research question?
2. Why is it important?
3. What are the problems solved by the chosen methods?
4. What is being measured?
5. How does the measurement help answer the research question?

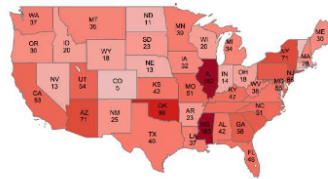


Figure 7: Introduced bills by state from ALEC model legislation

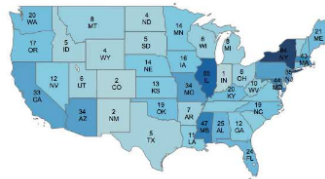


Figure 8: Introduced bills by state from ALICE model legislation

Outline

Document Distance

Preview: Retrieval-Augmented Generation

Clustering

Retrieval-Augmented Generation (RAG)

- ▶ Large language models (LLMs, covered later in the course) produce fluent text, but their knowledge is frozen at training time and they can **hallucinate** facts

Retrieval-Augmented Generation (RAG)

- ▶ Large language models (LLMs, covered later in the course) produce fluent text, but their knowledge is frozen at training time and they can **hallucinate** facts
- ▶ **Retrieval-Augmented Generation (RAG)** grounds the model in an external corpus: retrieve relevant documents, then let the LLM generate an answer conditioned on them

Retrieval-Augmented Generation (RAG)

- ▶ Large language models (LLMs, covered later in the course) produce fluent text, but their knowledge is frozen at training time and they can **hallucinate** facts
- ▶ **Retrieval-Augmented Generation (RAG)** grounds the model in an external corpus: retrieve relevant documents, then let the LLM generate an answer conditioned on them
- ▶ Two steps:
 1. **Retrieve**: given a query, find the most relevant documents in the corpus
 2. **Generate**: feed the query + retrieved documents to the LLM as context; it answers using that evidence (and can cite it)

Retrieval-Augmented Generation (RAG)

- ▶ Large language models (LLMs, covered later in the course) produce fluent text, but their knowledge is frozen at training time and they can **hallucinate** facts
- ▶ **Retrieval-Augmented Generation (RAG)** grounds the model in an external corpus: retrieve relevant documents, then let the LLM generate an answer conditioned on them
- ▶ Two steps:
 1. **Retrieve**: given a query, find the most relevant documents in the corpus
 2. **Generate**: feed the query + retrieved documents to the LLM as context; it answers using that evidence (and can cite it)
- ▶ **Key point**: the retrieval step is exactly the document-distance problem from today: vectorize query and corpus (L3), rank by cosine similarity (L4), return the top- k
 - ▶ “Sparse” retrieval = TF-IDF / BM25 – the elasticsearch workhorse from the Burgess et al. example
 - ▶ “Dense” retrieval with embeddings, and the pros and cons of RAG → Lecture 9

Outline

Document Distance

Preview: Retrieval-Augmented Generation

Clustering

K-Means Clustering

- ▶ **k-means clustering**: divide documents $\mathbf{x}$ into sets $S_1, \dots, S_k$ with associated centroids $\mu_1, \dots, \mu_k$; each doc is in set with closest centroid

K-Means Clustering

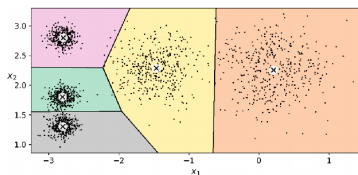
- ▶ **k-means clustering**: divide documents $\mathbf{x}$ into sets $S_1, \dots, S_k$ with associated centroids $\mu_1, \dots, \mu_k$; each doc is in set with closest centroid
- ▶ algorithm: initialize cluster centroids randomly, then shift around to minimize sum of within-cluster squared distance (features should be standardized)

$$\arg \min_S = \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \mu_i\|^2$$

K-Means Clustering

- ▶ **k-means clustering**: divide documents $\mathbf{x}$ into sets $S_1, \dots, S_k$ with associated centroids $\mu_1, \dots, \mu_k$; each doc is in set with closest centroid
- ▶ algorithm: initialize cluster centroids randomly, then shift around to minimize sum of within-cluster squared distance (features should be standardized)

$$\arg \min_S = \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} ||\mathbf{x} - \mu_i||^2$$



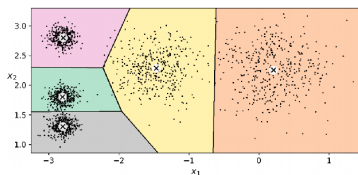
K-Means decision boundaries (Voronoi tessellation)

```
from sklearn.cluster import Kmeans
kmeans = KMeans(n_clusters=10)
kmeans.fit(X)
assigned_cluster = kmeans.labels_
```

K-Means Clustering

- ▶ **k-means clustering**: divide documents $\mathbf{x}$ into sets $S_1, \dots, S_k$ with associated centroids $\mu_1, \dots, \mu_k$; each doc is in set with closest centroid
- ▶ algorithm: initialize cluster centroids randomly, then shift around to minimize sum of within-cluster squared distance (features should be standardized)

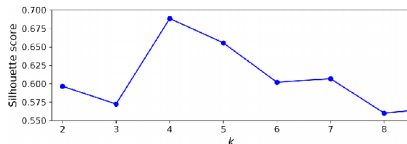
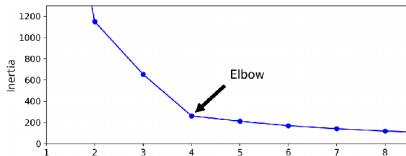
$$\arg \min_S \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \mu_i\|^2$$



K-Means decision boundaries (Voronoi tessellation)

```
from sklearn.cluster import Kmeans
kmeans = KMeans(n_clusters=10)
kmeans.fit(X)
assigned_cluster = kmeans.labels_
```

k (number of clusters) is the only hyperparameter, can select using:



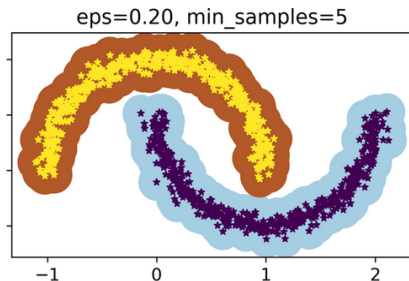
Other clustering algorithms

- ▶ “k-medoid” clustering use L1 distance rather than Euclidean distance; produces the “medoid” (median vector) for each cluster rather than “centroid” (mean vector)
 - ▶ less sensitive to outliers, and medoid can be used as representative data point

Other clustering algorithms

- ▶ “k-medoid” clustering use L1 distance rather than Euclidean distance; produces the “medoid” (median vector) for each cluster rather than “centroid” (mean vector)
 - ▶ less sensitive to outliers, and medoid can be used as representative data point

- ▶ **DBSCAN** defines clusters as continuous regions of high density
 - ▶ detects and excludes outliers automatically



- ▶ Agglomerative (hierarchical) clustering makes nested clusters

Note on terms and documents

- ▶ Recall that in $\mathbf{X}$,
 - ▶ each row / document $\mathbf{X}_{[d,:]}$ is a distribution over terms
 - ▶ each column / term $\mathbf{X}_{[:,w]}$ is a distribution over documents
- ▶ The same methods we used on the rows can be used on the columns:
 - ▶ apply cosine similarity to the columns to compare words (rather than compare documents)
 - ▶ apply k-means clustering to the columns to get clusters of similar words (rather than clusters of documents)